

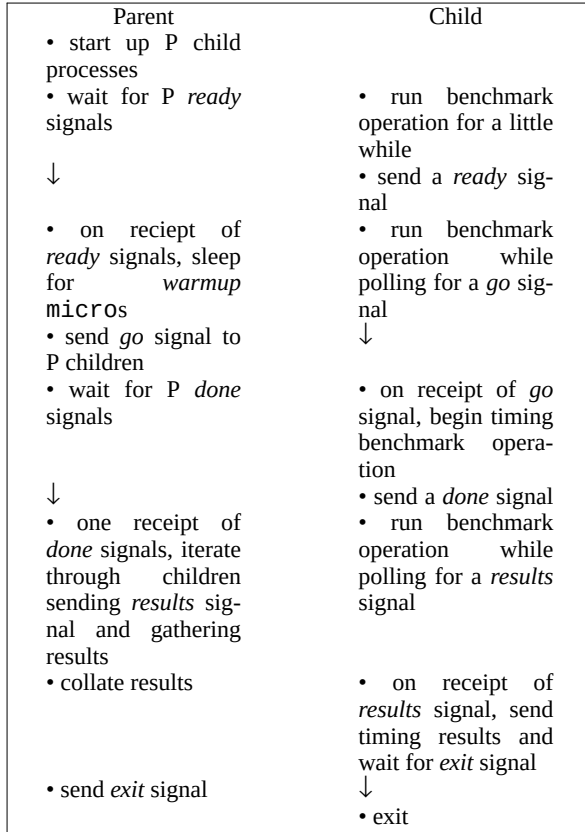


Table 1. Timing harness sequencing

Table 1 shows how the parent and child processes coordinate their activities to ensure that all children are actively running the benchmark activity while any child could be taking timing measurements.

The reason for the separate "exit" signal is to ensure that all properly managed children are alive until the parent allows them to die. This means that any SIGCHLD events that occur before the "exit" signal indicate a child failure.

3.3. Accuracy

The new timing harness also needs to ensure that the timing intervals are long enough for the results to be representative. The previous timing harness assumed that only single process results were important, and it was able to use timing intervals as short as possible while ensuring that errors introduced by the clock resolution were negligible. In many instances this meant that the timing intervals were smaller than a single scheduler time slice. The new timing harness must run benchmarked operations long enough to ensure that timing intervals are longer than a single scheduler time slice. Otherwise, you can get results which are complete nonsense. For example, running several copies of an `lmbench2` benchmark on a uni-processor machine will often report that the per-process performance with N jobs running in parallel is equivalent

to the performance with a single job running!¹

In addition, since the timing intervals now have to be longer than a single scheduler time slice, they also need to be long enough so that a single scheduler time slice is insignificant compared to the timing interval. Otherwise the timing results can be dramatically affected by small variations in the scheduler's behavior.

Currently `lmbench` does not measure the scheduler timeslice; the design blithely assumes that timeslices are generally on the order of 10-20ms, so one second timing intervals are sufficient. Some schedulers may utilize longer time slices, but this has not (yet) been a problem.

3.4. Resource consumption

One important design goal was that resource consumption be constant with respect to the number of child processes. This is why the harness uses shared pipes to communicate with the children, rather than having a separate set of pipes to communicate with each child. An early design of the system utilized a pair of pipes per child for communication and synchronization between the master and slave processes. However, as the number of child processes grew, the fraction of system resources consumed by the harness grew and the additional system overhead could start to interfere with the accuracy of the measurements.

Additionally, if the master has to poll (`select`) N pipes, then the system overhead of that operation also scales with the number of children.

3.5. Pipe atomicity

Since all communication between the master process and the slave (child) processes is done via a set of shared pipes, we have to ensure that we never have a situation where the message can be garbled by the intermingling of two separate messages from two separate children. This is ensured by either using pipe operations that are guaranteed to be atomic on all machines, or by coordinating between processes so that at most one process is writing at a time.

The atomicity guarantees are provided by having each client communicate synchronization states in one-byte messages. For example, the signals from the master to each child are one-byte messages, so each child only reads a single byte from the pipe. Similarly, the responses from the children back to the master are also one-byte messages. In this way no child can receive partial messages, and no message can be interleaved with any other message.

However, using this design means that we need to have a separate pipe for each *barrier* in the process, so

¹This was discovered by someone who naively attempted to parallelize `lmbench2` in this fashion, and I received a note from the dismayed developer describing the failed experiment.

the master uses three pipes to send messages to the children, namely: *start_signal*, *result_signal*, and *exit_signal*. If a single pipe was used for all three barrier events, then it is possible for a child to miss a signal, or if the signal is encoded into the message, then it is possible for a child to infinite loop pulling a signal off the pipe, recognizing that it has already received that signal so that it needs to push it back into the pipe, and then then re-receiving the same message it just re-sent.

However, all children share a single pipe to send data back to the master process. Usually the messages on this pipe are single-byte signals, such as *ready* or *done*. However, the timing data results need to be sent from the children to the master and they are (much) larger than a single-byte message. In this case, the timing harness sends a single-byte message on the *result_signal* channel, which can be received by at most one child process. This child then knows that it has sole ownership of the response pipe, and it writes its entire set of timing results to this pipe. Once the master has received all of the timing results from a single child, it sends the next one-byte message on the *result_signal* channel to gather the next set of timing results.

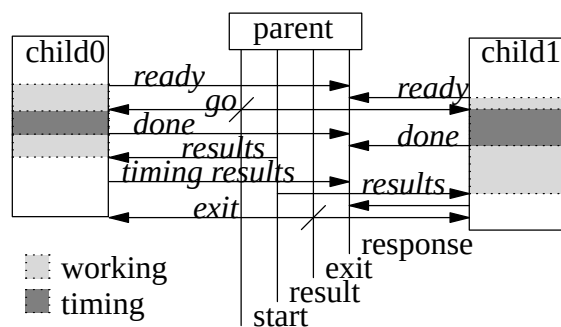


Figure 1. Control signals

The design of the signals is shown in Figure 1.

3.6. Benchmark initialization

By allowing the benchmark to specify an initialization routine that is run in the child processes, the new timing harness allows benchmarks to do either or both global initializations that are shared by all children and specific per-child initializations that are done independently by each child. Global initialization is done in the master process before the **benchmp** harness is called, so the state is preserved across the **fork** operations. Per-child initialization is done inside the **benchmp** harness by the optional initialization routine and is done after the **fork** operation.

Similarly, each benchmark is allowed to specify a cleanup routine that is run by the child processes just before exiting. This allows the benchmark routines to

release any resources that they may have used during the benchmark. Most system resources would be automatically released on process exit, such as file descriptors and shared memory segments, but some resources such as temporary files might need to be explicitly released by the benchmark.

3.7. Scheduler transients

Particularly on multi-processor systems, side-effects of process migration can dramatically affect program runtimes. For example, if the processes are all initially assigned to the same processor as the parent process, and the timing is done before the scheduler migrates the processes to other available processors, then the system performance will appear to be that of a uniprocessor. Similarly, if the scheduler is over-enthusiastic about re-assigning processes to processors, then performance will be worse than necessary because the processes will keep encountering cold caches and will pay exorbitant memory access costs.

The first case is a scheduler transient, and users may not want to measure such transient phenomena if their primary interest is in predicting performance for long-running programs. Conversely, that same user would be extraordinarily interested in the second phenomena. The harness was designed to allow users to specify that the benchmarked processes are run for long enough to (hopefully) get the scheduler past the transient startup phase, so it can measure the steady-state behavior.

3.8. Data analysis

Analyzing the data to produce representative results is a crucial step in the benchmarking process. **lmbench** generally reports the *median* result for 11 measurements. Most benchmarks report the results of a single measurement[8], an average of several results[11], or a trimmed mean[4].

Since **lmbench** is able to use timing intervals that are often smaller than a scheduler time slice when measuring single-process performance, the raw timing results are often severely skewed. Often most results cluster around a single value a small number of outliers with significantly larger values. The median is preferable to the mean when the data can be very skewed[9]. Since the timing intervals are significantly longer when the desired load is larger than a single process, the results tend not to be as badly skewed. In these cases we could use the *mean* instead, but we decide to use a uniform statistical framework, so we usually use the median.

In some instances, however, **lmbench** internally uses the *minimum* rather than the median, such as in **mh2**. In those instances, we are not trying to find the *representative* value, but rather the *minimum* value. There are only a few sources of error which could cause a the measured timing result to be shorter than the true elapsed time: the system clock is adjusted, or round-

off error in the clock resolution. The timing interval duration is set to ensure that the round-off error is bounded to 1% of the timing interval, and we blithely assume that people don't reset their system clocks while benchmarking their systems.

`lmbench` does not currently report any statistics representing measurement variation, such as the difference between the first and third quartiles. This is an enhancement under active consideration.

4. Interface

Unfortunately we had to move away from the macro-based timing harness used in `lmbench2` and migrate to a function-based system because the macros were too large for some C pre-processors.

```
typedef void (*bench_f)(iter_t iters,
                        void* cookie);
typedef void (*support_f)(void* cookie);

extern void benchmp(support_f initialize,
                   bench_f benchmark,
                   support_f cleanup,
                   int enough,
                   int parallel,
                   int warmup,
                   int repetitions,
                   void* cookie);

extern uint64 gettime();
extern uint64 get_n();
extern void nano(char* s, uint64 n);
extern void micro(char* s, uint64 n);
extern void mb(uint64 bytes);
```

Figure 2. Programming interface

Figure 2 shows the key elements of the new timing harness and result reporting interface. A brief description of the `benchmp` parameters:

enough

Enough can be used to ensure that a timing interval is at least 'enough' microseconds in duration. For most benchmarks this should be zero, but some benchmarks have to run for more time due to startup effects or other transient behavior.

parallel

is simply the number of instances of the benchmark that will be run in parallel on the system.

warmup

can be used to force the benchmark to run for warmup microseconds before the system starts making timing measurements. Note that it is a lower bound, not a fixed value, since it is simply the time that the parent sleeps after receiving the last "ready" signal from each child (and before it sends the "go" signal to the children).

repetitions

is the number of times the experiment should be repeated. The default is eleven.

cookie

is a pointer that can be used by the benchmark writer to pass in configuration information, such as buffer size or other parameters needed by the inner loop. In `lmbench3` it is generally used to point to a structure containing the relevant configuration information.

`gettime` returns the median timing interval duration, while `get_n` returns the number of iterations executed during that timing interval.

`nano` and `micro` print the passed string latency followed by the latency in terms of nanoseconds and microseconds respectively. The latency is computed as `gettime()/n`, where `n` is the passed parameter. The reason `n` is passed as a parameter is because the benchmark can actually execute the operation of interest multiple times during a single iteration. For example, the memory latency benchmarks typically repeat the memory load operation a hundred times inside the loop, so the actual number of operations is $100 \times \text{get_n}()$, and it is this value that should be passed to `nano` or `micro`.

`mb` reports the bandwidth in MB/s when given the total number of bytes processed during the timing interval. Note that for scalable benchmarks that process `size` bytes per iteration, the total number of bytes processed is $\text{get_n}() \times \text{parallel} \times \text{size}$.

```
#include "bench.h"

void
bench(iter_t iters, void* cookie)
{
    while (iters-- > 0) {
        getppid();
    }
}

int
main(int argc, char* argv[])
{
    benchmp(NULL, bench, NULL,
            0, 1, 0, TRIES, NULL);
    nano("getppid", get_n());
    return(0);
}
```

Figure 3. A sample benchmark

Figure 3 shows a sample benchmark that measures the latency of the `getppid` system call using this timing harness. Since there is no setup or cleanup needed for this benchmark, the *initialize* and *cleanup* parameters are NULL. The *bench* routine simply calls `getppid` as many times as requested, and the rest of the

parameters, *enough*, *parallel*, *warmup*, *repetitions*, and *cookie* are given with the default values.

5. Benchmarks

`lmbench` contains a large number of micro-benchmarks that measure various aspects of hardware and operating system performance. The benchmarks generally measure latency or bandwidth, but some new benchmarks also measure instruction-level parallelism.

Table 2 contains the full list of micro-benchmarks in `lmbench3`. Benchmarks that were converted to measure performance under scalable load are shown in *italics*, while the remaining benchmarks are shown with normal typeface. A detailed description of most benchmarks can be found in [12].

6. Scaling Benchmarks

There are a number of issues associated with converting single-process benchmarks with a single process to scalable benchmarks with several independent processes, in addition to the various issues addressed by the timing harness. Many of the benchmarks consume or utilize system resources, such as memory or network bandwidth, and a careful assessment of the likely resource contention issues is necessary to ensure that the benchmarks measure important aspects of system performance and not artifacts of artificial resource contention.

For example, the Linux 2.2 and 2.4 kernels use a single lock to control access to the kernel data structures for a file. This means that multiple processes accessing that file will have their operations serialized by that lock. If one is interested in how well a system can handle multiple independent accesses to separate files and if the child processes all access the same file, then this file sharing is an artificial source of contention with potentially dramatic effects on the benchmark results.

6.1. File System

A number of the benchmarks measure aspects of file system performance, such as `bw_file_rd`, `bw_mmap_rd`, `lat_mmap`, and `lat_pagefault`. It is not immediately apparent how these benchmarks should be extended to the parallel domain. For example, it may be important to know how file system performance scales when multiple processes are reading the same file, or when multiple processes are reading different files. The first case might be important for large, distributed scientific calculations, while the second might be more important for a web server.

However, for the operating system, the two cases are significantly different. When multiple processes access the same file, access to the kernel data structures for that file must be coordinated and so contention and locking of those structures can impact performance, while this is less true when multiple

Name	Measures
Bandwidth	
<i>bw_file_rd</i>	read and then load into processor
<i>bw_mem</i>	read, write, and copy data to/from memory
<i>bw_mmap_rd</i>	read from <code>mmap</code> 'ed memory
<i>bw_pipe</i>	<code>pipe</code> inter-process data copy
<i>bw_tcp</i>	TCP inter-process data copy
<i>bw_unix</i>	UNIX inter-process
Latency	
<i>lat_connect</i>	TCP connection
<i>lat_ctx</i>	context switch via <code>pipe</code> -based "hot-potato" token passing
<i>lat_dram_page</i>	DRAM page open
<i>lat_fcntl</i>	<code>fcntl</code> file locking "hot-potato" token passing
<i>lat_fifo</i>	FIFO "hot-potato" token passing
<i>lat_fs</i>	file creation and deletion
<i>lat_http</i>	http GET request latency
<i>lat_mem_rd</i>	memory read
<i>lat_mmap</i>	<code>mmap</code> operation
<i>lat_ops</i>	basic operations (<i>xor</i> , <i>add</i> , <i>mul</i> , <i>div</i> , <i>mod</i>) on (relevant) basic data types (<i>int</i> , <i>int64</i> , <i>float</i> , <i>double</i>)
<i>lat_pagefault</i>	page fault handler
<i>lat_pipe</i>	<code>pipe</code> "hot-potato" token passing
<i>lat_pmake</i>	time to complete <i>N</i> parallel jobs that do <i>usecs</i> -worth of work
<i>lat_proc</i>	procedure call overhead and process creation using <code>fork</code> , <code>fork</code> and <code>execve</code> , and <code>fork</code> and <code>/bin/sh</code>
<i>lat_rpc</i>	SUN RPC procedure call
<i>lat_select</i>	<code>select</code> operation
<i>lat_sem</i>	semaphore "hot-potato" token passing
<i>lat_sig</i>	signal handle installation and handling
<i>lat_syscall</i>	<code>open</code> , <code>close</code> , <code>getppid</code> , <code>write</code> , <code>stat</code> , <code>fstat</code>
<i>lat_tcp</i>	TCP "hot-potato" token passing
<i>lat_udp</i>	UDP "hot-potato" token passing
<i>lat_unix</i>	UNIX "hot-potato" token passing
<i>lat_unix_connect</i>	UNIX socket connection
<i>lat_usleep</i>	<code>usleep</code> , <code>select</code> , <code>pselect</code> , <code>nanosleep</code> , <code>setitimer</code> timer resolution
Other	
<i>disk</i>	zone bandwidths and seek times
<i>line</i>	cache line size
<i>lmdd</i>	<code>dd</code> clone
<i>par_mem</i>	memory subsystem ILP
<i>par_ops</i>	basic operation ILP
<i>stream</i>	STREAM clones
<i>tlb</i>	TLB size

Table 2. lmbench micro-benchmarks

processes access different files.

In addition, there are any number of issues associated with ensuring that the benchmarks are either measuring operating system overhead (e.g., that no I/O is actually done to disk), or actually measuring the